SUJET 2 : Développement d'une application mobile de découverte touristique	Concevoir une application Android permettant aux touristes de découvrir les sites touristiques.	Consultation des sites Géolocalisation Itinéraire Google Maps Favoris Partage sur réseaux sociaux	**KHADIM DRAME ET LAMINE FATY**

RÉPUBLIQUE DU SÉNÉGAL

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR, DE LA RECHERCHE ET DE L'INNOVATION

UNIVERSITÉ ASSANE SECK DE ZIGUINCHOR (UASZ)

UFR DES SCIENCES ET TECHNOLOGIES — DÉPARTEMENT INFORMATIQUE

Licence 3 - Informatique (L2I) — Année Universitaire 2025-2026

MODULE D'ENSEIGNEMENT :

DÉVELOPPEMENT D'APPLICATIONS ORIENTÉES SERVICES

COMPTE-RENDU TECHNIQUE DE TRAVAUX PRATIQUES (DEVOIR NOTÉ)

CONCEPTION ET DÉPLOIEMENT D'UNE ARCHITECTURE MICROSERVICES DISTRIBUÉE

Transition d'un socle monolithique vers un écosystème découplé : Spring Data JPA, DTOs & MapStruct, Communication Synchrone OpenFeign et Conteneurisation Docker

Présenté par le binôme :

- **Collé SYLL**
  Matricule : 202300489
- **Ndeye Thiamba FAYE**
  Matricule : 202300488

Sous la direction de :

Dr Lamine FATY
Enseignant-Chercheur
Département Informatique, UASZ

Ziguinchor, Sénégal — Février / Mars 2026

SOMMAIRE DU COMPTE-RENDU

I. INTRODUCTION GÉNÉRALE **3**

1.1 Contexte pédagogique et problématique du socle monolithique **3**

1.2 Pourquoi abandonner le monolithe : limites et goulets d'étranglement **3**

1.3 Objectifs d'apprentissage et articulation des Travaux Pratiques (TP 1, TP 2, TP 3) **4**

II. DÉVELOPPEMENT TECHNIQUE & IMPLÉMENTATION DES MICROSERVICES **5**

2.1 Cartographie architecturale de l'écosystème distribué **5**

2.2 TP 1 : Couche de persistance avancée avec Spring Data JPA **6**

- Entité relationnelle 'Patrimoine.java' (Code commenté) **6**
- Interface de persistance 'PatrimoineRepository.java' (Slide 16 TP1) **7**

2.3 TP 2 : Isolation par DTO, Génération MapStruct & Communication OpenFeign **8**

- Le pattern DTO et découplage du modèle physique (Slide 7 TP2) **8**
- Génération automatique de conversion avec MapStruct (Slide 9 TP2) **9**
- Exposition REST et Contrôleur d'API (Slides 17-21 TP2) **10**
- Communication inter-services synchrone avec OpenFeign (Slide 32 TP2) **11**
- Application stricte de la règle d'isolation du Slide 28 **12**

2.4 TP 3 : Conteneurisation Docker, Packaging et Orchestration Docker Compose **13**

- Fichier Dockerfile multi-services (Java 21 LTS) **13**
- Orchestration multi-conteneurs et intégration MySQL XAMPP **14**
- Nomenclature des images et exécution opérationnelle **15**

III. CONCLUSION GÉNÉRALE & BILAN TECHNIQUE **16**

3.1 Tableau comparatif : Architecture Monolithique vs Microservices Distribués **16**

3.2 Synthèse des compétences acquises et perspectives d'industrialisation **17**

📌 **GLOSSAIRE DES ACRONYMES TECHNIQUES :** *JPA : Java Persistence API | ORM : Object-Relational Mapping | DTO : Data Transfer Object | OpenFeign : Client REST déclaratif Spring Cloud | CRUD : Create, Read, Update, Delete | SOA : Service-Oriented Architecture | REST : Representational State Transfer | JSON : JavaScript Object Notation.*

I. INTRODUCTION GÉNÉRALE

1.1 Contexte Pédagogique et Périmètre du Module

Le présent travail s'inscrit dans le cadre des travaux pratiques et de l'évaluation semestrielle du module « Développement d'Applications Orientées Services » (Licence 3 Informatique - L2I) dispensé par Dr Lamine FATY à l'Université Assane SECK de Ziguinchor. L'enjeu fondamental de cet enseignement est d'inculquer aux futurs ingénieurs les paradigmes modernes de conception logicielle distribuée, en rupture avec les architectures monolithiques traditionnelles.

Pour illustrer concrètement ces concepts, notre binôme (Collé SYLL & Ndeye Thiamba FAYE) a pris comme point de départ une application patrimoniale et touristique initialement développée sous forme d'un bloc monolithique indivisible. À travers une série méthodique de trois Travaux Pratiques (TP 1, TP 2 et TP 3), nous avons décomposé ce bloc pour implémenter une architecture orientée microservices moderne, hautement disponible et conteneurisée.

1.2 Pourquoi Abandonner le Monolithe ? Analyse Critique des Limites

L'architecture monolithique réunit au sein d'un unique exécutable (un gros fichier JAR ou WAR) l'ensemble des modules fonctionnels : gestion des sites de patrimoine, gestion des circuits touristiques, gestion des événements, réservations et interface utilisateur. Si cette approche offre une simplicité trompeuse lors de la phase initiale de prototypage, elle révèle rapidement des faiblesses rédhibitoires à l'échelle industrielle :

1. Couplage fort et point unique de défaillance (Single Point of Failure) : Toutes les fonctionnalités s'exécutent dans la même JVM et partagent le même schéma de base de données relationnelle. Une exception non gérée ou une fuite de mémoire dans le module de gestion des photos de patrimoine entraîne le crash immédiat de l'ensemble du système (y compris les réservations en ligne).

2. Incapacité à passer à l'échelle de façon granulaire (Scalabilité inefficace) : Même si seule la recherche de patrimoine connaît un pic d'affluence le week-end, il est impossible de ne répliquer que cette fonction. Il faut dupliquer l'intégralité du monolithe, gaspillant ainsi des ressources processeur et mémoire substantielles.

3. Cycle de vie et déploiements rigides : La moindre modification d'une ligne de code (par exemple ajuster le format de date d'un événement) exige de recompiler, retester et redéployer la totalité de l'application avec un arrêt de service préjudiciable.

> 📌 **NOTE D'INGÉNIERIE LOGICIELLE :** *La transition vers les microservices consiste à découper le système en services autonomes, chacun responsable d'un domaine métier précis (Bounded Context), disposant de son propre modèle de données, communicant par des protocoles légers (REST/JSON) et pouvant être déployé indépendamment.*

1.3 Objectifs d'Apprentissage et Articulation des 3 TP

La progression pédagogique suivie tout au long de nos séances de travaux pratiques s'articule autour de trois jalons complémentaires :

• **Travail Pratique n°1 (Persistance & ORM) :** Mettre en place la couche d'accès aux données avec Spring Data JPA au sein des entités métiers, éliminant les requêtes SQL natives verbeuses au profit des repositories managés et du mapping objet-relationnel Hibernate.

• **Travail Pratique n°2 (DTO, MapStruct & OpenFeign) :** Mettre en œuvre le pattern DTO pour sécuriser et découpler les entités de base de données de l'API publique, automatiser les conversions à la compilation grâce à MapStruct, et

établir une communication synchrone inter-services déclarative avec Spring Cloud OpenFeign sans créer de dépendances croisées.

- **Travail Pratique n°3 (Docker & Déploiement Orchestré) :** Industrialiser le système en conteneurisant chaque microservice dans une image Docker dédiée (étiquetée 'collesyll/*:v1'), et orchestrer l'ensemble du cluster via Docker Compose avec intégration réseau transparente vers la base MySQL XAMPP de l'hôte.

II. DÉVELOPPEMENT TECHNIQUE & IMPLÉMENTATION DES MICROSERVICES

2.1 Cartographie Architecturale de l'Écosystème Distribué

L'architecture cible déploie 5 microservices autonomes conçus avec Spring Boot 3 et Java 21 LTS, chacun encapsulant son périmètre fonctionnel et ses propres endpoints :

Microservice	Port Hôte / Conteneur	Image Docker	Responsabilité Métier
ms-patrimoine	8081 / 3001	`collesyll/patrimoines:v1`	Gestion CRUD des patrimoines, sites culturels et géolocalisation.
ms-circuit	8082 / 3002	`collesyll/circuits:v1`	Gestion des circuits, des étapes et enrichissement OpenFeign.
ms-evenement	8083 / 3003	`collesyll/evenements:v1`	Gestion des manifestations, dates et festivités régionales.
ms-reservation	8084 / 3004	`collesyll/reservations:v1`	Gestion des réservations touristiques et suivi des demandes.
patrimoine-touristique-web	8080 / 8080	`collesyll/patrimoine-web:v1`	Portail Web frontend (Thymeleaf/Bootstrap) pour les usagers.

2.2 TP 1 : Couche de Persistance Avancée avec Spring Data JPA

Le Travail Pratique n°1 a posé les fondations du stockage relationnel. Plutôt que d'écrire des requêtes SQL bas niveau susceptibles d'introduire des failles d'injection et de briser la portabilité, nous avons utilisé Spring Data JPA reposant sur l'ORM Hibernate.

L'entité centrale 'Patrimoine' du microservice 'ms-patrimoine' illustre le mapping relationnel avec les annotations JPA Jakarta et les annotations Lombok (@Data, @NoArgsConstructor, @AllArgsConstructor, @Builder) réduisant drastiquement le code boilerplate :

💻 **Code source extrait du projet — `ms-patrimoine/.../model/Patrimoine.java`**

```
package sn.zig.patrimoine.model;

import jakarta.persistence.*;
import lombok.*;

/**
 * Entité JPA Patrimoine du microservice ms-patrimoine (TP1 & TP2 Dr Lamine FATY).
 * Modèle relationnel mappé avec la table 'patrimoines'.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
@Table(name = "patrimoines")
public class Patrimoine {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "code")
    private String codePatrimoine;

    @Column(name = "nom", nullable = false)
    private String nomPatrimoine;

    @Lob
    @Column(name = "description", columnDefinition = "TEXT")
    private String descriptionPatrimoine;

    @Lob
    @Column(name = "historique", columnDefinition = "TEXT")
    private String historiquePatrimoine;

    @Column(name = "adresse")
    private String adressePatrimoine;

    @Column(name = "capacite")
    private int capacitePatrimoine;

    @Column(name = "latitude")
    private Double latitudePatrimoine;

    @Column(name = "longitude")
    private Double longitudePatrimoine;
}
```

Pour interagir avec la table 'patrimoines', l'interface 'PatrimoineRepository' étend 'JpaRepository<Patrimoine, Long>'. Conformément au Slide 16 du cours de Dr Lamine FATY, cette déclaration confère immédiatement toutes les opérations CRUD standard (save, findById, findAll, deleteById, count) sans rédiger la moindre ligne d'implémentation SQL :

💻 **Code source extrait du projet — `ms-patrimoine/.../repository/PatrimoineRepository.java`**

```
package sn.zig.patrimoine.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import sn.zig.patrimoine.model.Patrimoine;
import java.util.Optional;

/**
 * Interface Repository JPA pour ms-patrimoine (Slide 16 du TP1).
 * Fournit les opérations CRUD et requêtes dérivées managées par Hibernate.
 */
@Repository
public interface PatrimoineRepository extends JpaRepository<Patrimoine, Long> {

    // Méthode dérivée générée automatiquement par Spring Data JPA
    Optional<Patrimoine> findByCodePatrimoine(String codePatrimoine);
```

```
}
```

2.3 TP 2 : Isolation par DTO, Génération MapStruct & Communication OpenFeign

A. Le Pattern DTO (Data Transfer Object) — Slide 7

Dans une architecture orientée services propre, exposer directement les entités JPA (@Entity) aux clients externes via les API REST constitue une grave violation des bonnes pratiques de génie logiciel :

1. Couplage fort entre persistance et exposition : Une modification de structure de base de données (ajout ou renommage d'une colonne) casse immédiatement tous les clients consommateurs si l'entité est exposée sans filtre.

2. Vulnérabilité de sécurité : L'exposition directe risque de divulguer des attributs confidentiels ou des identifiants internes non pertinents pour l'utilisateur final.

3. Problème de sérialisation : Le chargement paresseux (Lazy Loading) d'entités liées provoque fréquemment des erreurs 'LazyInitializationException' ou des boucles infinies de sérialisation JSON (références circulaires).

💻 **Code source extrait du projet — `ms-patrimoine/.../dto/PatrimoineDto.java`**

```java
package sn.zig.patrimoine.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

/**
 * Data Transfer Object pour Patrimoine (TP 2 - Slide 7 Dr Lamine FATY).
 * Rôles : contrat d'échange public, découplage des couches et sécurité.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class PatrimoineDto {

    private Long id;
    private String nomPatrimoine;
    private String codePatrimoine;
    private String descriptionPatrimoine;
    private String historiquePatrimoine;
    private String adressePatrimoine;
    private int capacitePatrimoine;
    private Double latitudePatrimoine;
    private Double longitudePatrimoine;
}
```

B. Génération Automatique des Mappings avec MapStruct — Slide 9

La conversion manuelle entre Entité et DTO (via des enchaînements verbeux de 'dto.setNom(entity.getNom())') est source de bugs et nuit à la lisibilité. Conformément au Slide 9 du TP2, nous avons intégré la bibliothèque MapStruct. Fonctionnant comme processeur d'annotations au moment de la compilation javac, MapStruct génère un code Java natif ultra-performant et type-safe.

💻 **Code source extrait du projet — `ms-patrimoine/.../mappers/PatrimoineMapper.java`**

```java
package sn.zig.patrimoine.mappers;

import org.mapstruct.Mapper;
import sn.zig.patrimoine.dto.PatrimoineDto;
import sn.zig.patrimoine.model.Patrimoine;

/**
 * Interface Mapper MapStruct (TP 2 - Slide 9 Dr Lamine FATY).
 * Génère automatiquement le code de conversion bidirectionnelle à la compilation.
 */
@Mapper(componentModel = "spring")
public interface PatrimoineMapper {

    // Convertit le DTO reçu du contrôleur en Entité pour la persistance
    Patrimoine toEntity(PatrimoineDto patrimoineDto);

    // Convertit l'Entité extraite de la base en DTO pour la réponse HTTP
    PatrimoineDto toDto(Patrimoine patrimoine);
}
```

C. Contrôleur REST et Exposition des Endpoints — Slides 17 à 21

Le composant 'PatrimoineController' expose les opérations CRUD standard sur les DTOs, avec injection par constructeur gérée par Lombok (@RequiredArgsConstructor) :

💻 **Code source extrait du projet — `ms-patrimoine/.../controller/PatrimoineController.java`**

```java
package sn.zig.patrimoine.controller;

import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import sn.zig.patrimoine.dto.PatrimoineDto;
import sn.zig.patrimoine.service.IPatrimoineService;
import java.util.List;

/**
 * Contrôleur REST pour ms-patrimoine avec DTO (TP 2 - Slides 17 à 21 Dr Lamine FATY).
 */
@RestController
@RequestMapping("/api/patrimoines")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class PatrimoineController {

    private final IPatrimoineService patrimoineService;

    // 1. Récupérer tous les patrimoines (Slide 17)
    @GetMapping
    public List<PatrimoineDto> getAllPatrimoines() {
        return this.patrimoineService.getAllPatrimoines();
    }

    // 2. Récupérer un patrimoine par son ID (Slide 18)
```

```
    @GetMapping("/{id}")
    public ResponseEntity<PatrimoineDto> getPatrimoineById(@PathVariable("id") Long id) {
        try {
            return ResponseEntity.ok(this.patrimoineService.getByIdPatrimoine(id));
        } catch (Exception e) {
            return ResponseEntity.notFound().build();
        }
    }

    // 3. Créer un patrimoine (Slide 19)
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public PatrimoineDto createPatrimoine(@RequestBody PatrimoineDto patrimoineDto) {
        return this.patrimoineService.savePatrimoine(patrimoineDto);
    }
}
```

D. Communication Synchrone Inter-Services avec OpenFeign — Slide 32

Un cas d'usage central de l'architecture distribuée est la collaboration entre services : le microservice 'ms-circuit' gère des parcours touristiques composés d'étapes. Chaque étape référence l'identifiant d'un patrimoine ('patrimoineId') stocké dans 'ms-patrimoine'.

Dans l'ancien modèle monolithique, une simple jointure SQL SQL (JOIN) résolvait le problème. Dans notre architecture microservices, 'ms-circuit' n'a pas accès à la base de données de 'ms-patrimoine'. Pour enrichir les données du circuit, nous avons utilisé Spring Cloud OpenFeign, un client HTTP déclaratif qui transforme une interface annotée en appel REST transparent :

💻 **Code source extrait du projet — `ms-circuit/.../client/PatrimoineClient.java`**

```
package sn.zig.circuit.client;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import sn.zig.circuit.dto.PatrimoineDto;
import java.util.List;

/**
 * Feign API Client pour interroger ms-patrimoine (TP 2 - Slide 32 Dr Lamine FATY).
 * Permet à ms-circuit d'enrichir ses circuits sans requête SQL directe.
 */
@FeignClient(name = "ms-patrimoine", url = "${ms.patrimoine.url:http://localhost:8081}")
public interface PatrimoineClient {

    // Appel REST synchrone : GET http://ms-patrimoine:8081/api/patrimoines/{id}
    @GetMapping("/api/patrimoines/{id}")
    PatrimoineDto getPatrimoineById(@PathVariable("id") Long id);

    @GetMapping("/api/patrimoines")
    List<PatrimoineDto> getAllPatrimoines();
}
```

> 📌 **APPLICATION DE LA RÈGLE D'ISOLATION DU SLIDE 28 :** *Comme l'impose le Slide 28 du cours de Dr Lamine*

> *FATY, le microservice 'ms-circuit' définit son PROPRE 'PatrimoineDto' (sn.zig.circuit.dto.PatrimoineDto) plutôt que d'importer le package de 'ms-patrimoine'. Cette règle empêche tout couplage binaire : si 'ms-patrimoine' évolue en interne, 'ms-circuit' reste intact tant que le contrat JSON est respecté.*

Voici comment 'CircuitServiceImpl' utilise ce client Feign pour enrichir les circuits et sécuriser les éventuelles pannes temporaires grâce à la méthode 'fetchPatrimoineSafely' :

💻 Code source extrait du projet — `ms-circuit/.../service/impl/CircuitServiceImpl.java`

```
// Extrait de CircuitServiceImpl.java dans ms-circuit
@Override
@Transactional(readOnly = true)
public List<PatrimoineDto> getPatrimoinesDuCircuit(Long circuitId) {
    List<Etape> etapes = etapeRepository.findByCircuitIdOrderByPositionAsc(circuitId);
    List<PatrimoineDto> patrimoines = new ArrayList<>();

    for (Etape etape : etapes) {
        // Appel synchrone OpenFeign sécurisé vers ms-patrimoine
        PatrimoineDto patrimoine = fetchPatrimoineSafely(etape.getPatrimoineId());
        if (patrimoine != null) {
            patrimoines.add(patrimoine);
        }
    }
    return patrimoines;
}

private PatrimoineDto fetchPatrimoineSafely(Long patrimoineId) {
    try {
        return patrimoineClient.getPatrimoineById(patrimoineId);
    } catch (Exception e) {
        // En cas d'indisponibilité temporaire du microservice ms-patrimoine
        return PatrimoineDto.builder()
                .id(patrimoineId)
                .nomPatrimoine("Patrimoine #" + patrimoineId + " (Indisponible)")
                .descriptionPatrimoine("Information temporairement indisponible")
                .build();
    }
}
```

2.4 TP 3 : Conteneurisation Docker, Packaging et Orchestration Docker Compose

A. Fichier Dockerfile Multi-Services (Java 21 LTS)

Pour garantir qu'une application fonctionne de manière identique quel que soit l'ordinateur ou le système d'exploitation cible, le TP 3 standardise le déploiement par la conteneurisation. Chaque microservice dispose de son propre fichier 'Dockerfile' basé sur une image officielle OpenJDK 21 :

💻 Code source extrait du projet — `ms-patrimoine/Dockerfile`

```
# Image de base officielle Java 21 LTS légère
FROM eclipse-temurin:21-jdk

# Métadonnées du binôme (Matricules L3 L2I)
LABEL maintainer="Collé SYLL (202300489) & Ndeye Thiamba FAYE (202300488)"
```

```
# Port d'écoute interne du microservice
EXPOSE 8081

# Copie du binaire JAR compilé par Maven
COPY target/*.jar app.jar

# Commande d'exécution au lancement du conteneur
ENTRYPOINT ["java", "-jar", "app.jar"]
```

B. Orchestration Multi-Conteneurs et Intégration MySQL XAMPP

L'orchestration des 5 microservices est centralisée dans le fichier 'docker-compose.yml'. Un défi technique majeur résolu lors du TP 3 a été de faire communiquer les conteneurs Docker isolés avec le serveur de base de données MySQL hébergé sur la machine physique hôte (via XAMPP sur le port 3306).

Grâce à l'instruction 'extra_hosts: ["host.docker.internal:host-gateway"]' et à la chaîne de connexion 'jdbc:mysql://host.docker.internal:3306/patrimoinedb', tous les microservices accèdent avec succès à la base de données sans perturber l'installation locale :

💻 **Code source extrait du projet — `docker-compose.yml`**

```
services:
  # 1. Microservice ms-patrimoine (TP 2 & TP 3)
  ms-patrimoine:
    build:
      context: ./ms-patrimoine
      dockerfile: Dockerfile
    image: collesyll/patrimoines:v1
    container_name: ms-patrimoine-container
    ports:
      - "3001:8081"
      - "8081:8081"
    environment:
      SPRING_DATASOURCE_URL:
jdbc:mysql://host.docker.internal:3306/patrimoinedb?useSSL=false&serverTimezone=UTC&allowPublicK
eyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: ""
      SERVER_PORT: 8081
    extra_hosts:
      - "host.docker.internal:host-gateway"

  # 2. Microservice ms-circuit (OpenFeign vers ms-patrimoine)
  ms-circuit:
    build:
      context: ./ms-circuit
      dockerfile: Dockerfile
    image: collesyll/circuits:v1
    container_name: ms-circuit-container
    depends_on:
      - ms-patrimoine
    ports:
      - "3002:8082"
      - "8082:8082"
    environment:
      SPRING_DATASOURCE_URL:
jdbc:mysql://host.docker.internal:3306/patrimoinedb?useSSL=false&serverTimezone=UTC&allowPublicK
```

```
eyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: ""
      SERVER_PORT: 8082
      MS_PATRIMOINE_URL: http://ms-patrimoine:8081
    extra_hosts:
      - "host.docker.internal:host-gateway"

  # 3. Microservice ms-evenement
  ms-evenement:
    build: { context: ./ms-evenement, dockerfile: Dockerfile }
    image: collesyll/evenements:v1
    container_name: ms-evenement-container
    ports: [ "3003:8083", "8083:8083" ]
    extra_hosts: [ "host.docker.internal:host-gateway" ]

  # 4. Microservice ms-reservation
  ms-reservation:
    build: { context: ./ms-reservation, dockerfile: Dockerfile }
    image: collesyll/reservations:v1
    container_name: ms-reservation-container
    ports: [ "3004:8084", "8084:8084" ]
    extra_hosts: [ "host.docker.internal:host-gateway" ]

  # 5. Portail Web Frontend Touriste & Conservateur
  patrimoine-touristique-web:
    build: { context: ./patrimoine-touristique-web, dockerfile: Dockerfile }
    image: collesyll/patrimoine-web:v1
    container_name: patrimoine-web-container
    ports: [ "8080:8080" ]
    extra_hosts: [ "host.docker.internal:host-gateway" ]
```

C. Cycle Opérationnel de Déploiement

Le cycle complet de packaging et de démarrage du cluster distribué s'exécute en deux commandes standards :

1. Compilation et empaquetage Maven : mvn clean package -DskipTests (Génération des fichiers JAR dans chaque dossier target)

2. Construction et instanciation Docker : docker-compose up --build -d (Construction des images 'collesyll/*:v1' et démarrage en arrière-plan)

3. Contrôle de santé des conteneurs : docker-compose ps (Vérification du statut 'Up' et des ports exposés)

III. CONCLUSION GÉNÉRALE & BILAN TECHNIQUE

3.1 Tableau Comparatif : Monolithe vs Architecture Microservices

Au terme de cette transition architecturale, le tableau synthétique ci-dessous met en lumière les gains qualitatifs majeurs apportés par les microservices par rapport au monolithe d'origine :

Critère de Génie Logiciel	Ancienne Approche (Monolithe)	Nouvelle Approche (Microservices TP1-TP3)
Découplage & Autonomie	Fort couplage interne, base partagée unique.	Services autonomes, contrats d'API découplés par DTO.
Performance & Mapping	Conversions manuelles ou jointures SQL lourdes.	MapStruct ultra-rapide à la compilation, OpenFeign synchrone.
Résilience & Tolérance aux pannes	Un crash dans un module fait tomber tout le site.	Pannes isolées : fallback local si un service est inaccessible.
Scalabilité & Déploiement	Déploiement lourd et intégral du JAR/WAR.	Scalabilité granulaire service par service avec Docker Compose.
Portabilité de l'environnement	Dépend de la configuration de la machine hôte.	Conteneurs standardisés 'collesyll/*:v1' (Java 21 LTS).

3.2 Synthèse des Compétences Acquises & Perspectives

La réalisation de ces Travaux Pratiques a permis à notre binôme (Collé SYLL & Ndeye Thiamba FAYE) de maîtriser l'ensemble de la chaîne de valeur du développement orienté services : de la persistance relationnelle Spring Data JPA jusqu'à l'orchestration multi-conteneurs Docker, en passant par le découplage DTO/MapStruct et la communication synchrone déclarative OpenFeign.

Les défis majeurs rencontrés — notamment la résolution des adresses réseau entre conteneurs Docker et la base de données XAMPP locale ('host.docker.internal'), ainsi que la stricte application de la règle d'isolation du Slide 28 sans dépendances croisées — ont renforcé notre compréhension pratique des architectures distribuées modernes.

Comme perspectives d'industrialisation futures, l'architecture mise en place pourra être enrichie d'une Spring Cloud Gateway pour unifier les points d'entrée externes, d'un registre de services Eureka pour la découverte dynamique des instances, et d'un serveur d'autorisation Keycloak pour sécuriser l'ensemble des échanges par des jetons JWT.